



THE UNIVERSITY OF
SYDNEY

The University of Sydney

School of Computer Science

Assignment 2 Part 2: Report

Student 1: 540493238

Student 2: 530524801

Student 3: 520523553

An Assignment submitted for the UoS:

COMP3308 Introduction to Artificial Intelligence

May 17, 2026

Contents

1	Introduction	2
2	Data	2
3	The KNN+ and DT+ Algorithms	3
3.1	KNN+	3
3.2	DT+	3
4	Results and Discussion	5
4.1	Results	5
4.2	Discussion	5
4.2.1	Overall classifier performance	5
4.2.2	Effect of modified algorithms (Comparison of MyKNN with MyKNN+ and MyDT with MyDT+)	6
4.2.3	Comparison with Weka	7
4.2.4	Effect of pruning in Weka J48 (DT-unpruned and DT-pruned)	7
4.2.5	Comparison of Weka tree based classifiers -single tree and ensemble of trees (DT-pruned, Bagging, Boosting and Random Forest)	8
5	Conclusion	9
6	Reflection	10
7	Acknowledgement of AI use	11
8	Appendix	12
8.1	Hyperparameter Sweeps	12
8.2	Diagrams of Decision Trees	12
8.3	Python Implementation of KNN+ and DT+	16

1 Introduction

This study aims to evaluate and compare different machine learning classifiers for predicting the survival outcomes of patients with heart failure. The task is a binary classification problem, where each patient is classified as either died or "survived" based on a set of personal and clinical attributes. In this study, we implemented two standard classifiers K-Nearest Neighbour (KNN) and Decision Tree (DT) from scratch. We also developed modified versions of these algorithms, referred to as KNN+ and DT+. These classifiers were evaluated using stratified 10-fold cross-validation and then compared with a range of classifiers available in Weka.

This study is important because heart failure is a serious medical condition, and analysing patient data may help reveal patterns associated with survival outcomes. More broadly, the task provides a useful case study of how machine learning can be applied to health-related data. It is also interesting from a methodological perspective because the dataset is relatively small, fully categorical, and class-imbalanced, which makes classifier behaviour more informative than accuracy alone. Comparing different algorithms and their modified versions therefore allows us to examine not only which methods perform best overall, but also how different modelling choices affect the detection of the clinically important "died" class.

2 Data

The data set used in this study is a modified version of the Heart Failure dataset. It contains 273 patient records, where each record represents one patient. Each patient is described by 11 nominal attributes, age, anaemia, CPK, diabetes, ejection fraction, high blood pressure, platelets, serum creatinine, serum sodium, sex and smoking status. The class attribute represents the patient's outcome during the follow up period and has two possible values: "died" and "survived". All attributes are categorical rather than continuous, the classifiers compare patients based on their attribute values rather than raw numerical measurements.

The dataset is also class-imbalanced. Out of the 273 examples, 90 belong to the class "died" and 183 belong to the class "survived". This means that the majority class is "survived". Because of this imbalance, accuracy alone is not sufficient to evaluate classifier performance. A model can obtain reasonable accuracy by simply predicting the majority class more often, but still performing poorly on the more important minority class. Hence, we also considered precision, recall, and F_1 -score (F -measure) for the "died" class in our evaluation.

In general the dataset is suitable for performing tabular classification because all features are already discretised/nominal, which matches the assumptions of both KNN for nominal data and the Decision Tree classifier used in this study.

3 The KNN+ and DT+ Algorithms

Let the training set be $D = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ denotes the i -th training example described by d nominal attributes, and $y^{(i)} \in \{\text{died}, \text{survived}\}$ is its class label.

3.1 KNN+

Our baseline KNN treats all attributes equally and gives all k neighbours the same voting influence. We modified this into two ways.

First, KNN+ uses a **feature-weighted distance**. The motivation is that some attributes appear to be more informative than others for distinguishing between the two classes, so mismatches on those attributes should contribute more to the distance. For each feature, we estimate a simple importance score from the training data based on how strongly its values are associated with class differences. These scores are then normalised and used as feature weights. For a test example $\mathbf{x}$ and a training example $\mathbf{x}^{(i)}$, KNN+ computes a weighted nominal distance of the form

$$d(\mathbf{x}, \mathbf{x}^{(i)}) = \sum_{j=1}^d w_j \mathbf{1}\{x_j \neq x_j^{(i)}\},$$

where w_j is the learned weight for feature j , and $\mathbf{1}$ is the indicator function.

Second, after selecting the k nearest neighbours, KNN+ uses **distance=weighted voting** rather than ordinary majority voting. A closer neighbour should have more influence on the prediction than a more distant neighbour, so the voting weight for neighbour i is defined as

$$\alpha_i = \frac{1}{d(\mathbf{x}, \mathbf{x}^{(i)}) + \varepsilon},$$

where adding the constant $\varepsilon > 0$ avoids dividing by zero, we chose $\varepsilon = 1$ in our implementation. The class scores are then obtained by summing neighbour weights within each class:

$$S_{\text{died}} = \sum_{i \in \mathcal{N}_k(\mathbf{x})} \alpha_i \mathbf{1}\{y^{(i)} = \text{died}\}, \quad S_{\text{survived}} = \sum_{i \in \mathcal{N}_k(\mathbf{x})} \alpha_i \mathbf{1}\{y^{(i)} = \text{survived}\}.$$

The final prediction is the class with larger score, and in the case of a tie, the prediction is "died".

Thus, by weighting both features and neighbour votes, KNN+ makes the classifier more sensitive to the most relevant local structure in data.

3.2 DT+

Our baseline decision tree uses **Information Gain** to choose the splitting attribute and grows until no further standard split is possible. This can produce very small and highly specific leaves, especially on a relatively small categorical dataset. Such leaves may overfit as they might fit the training data well but generalise poorly to unseen examples.

To address this, we designed DT+ as a **minimum-leaf-size decision tree**. The key idea is that a split should only be allowed if it does not create very small child nodes. More precisely, suppose that at a node we consider splitting on attribute A_j , producing child subsets $D_t^{(v)}$ for each value $v \in \text{Values}(A_j)$. In DT+, this split is considered valid only if every resulting child node satisfies $|D_t^{(v)}| \geq m$, where m is a fixed minimum leaf size. In our experiments, this hyperparameter was tuned empirically and the final DT+ used the setting that gave the best overall performance.

Among all valid splits, DT+ still chooses the attribute with the highest Information Gain

$$\text{IG}(D_t, A_j) = H(D_t) - \sum_{v \in \text{Values}(A_j)} \frac{|D_t^{(v)}|}{|D_t|} H(D_t^{(v)}),$$

where $H(D_t)$ is the entropy of the class labels at node t . If no valid split exists, the node becomes a leaf and is labelled with the majority class among the training examples reaching that node. As in the baseline tree, ties are resolved in favour of "died".

Compared to the baseline unpruned tree, this produces a more constrained model and was expected to improve generalisation, although, as shown later in the results, this also introduced a trade-off between overall accuracy and sensitivity to the "died" class.

4 Results and Discussion

4.1 Results

The classifiers were evaluated using stratified 10 fold cross validation. Table 1 reports the accuracy of all classifiers, together with precision, recall and F_1 -score for the died class. The died class was selected for the additional measures because identifying patients who died is important in this prediction task. For MyKNN, $k = 3$ was selected as a recall focused setting: although $k = 1$ achieved the highest recall for the died class, its accuracy and precision were much lower, while $k = 3$ provided a less extreme trade off. MyKNN+ used $k = 7$ because it provided stronger overall performance for the modified KNN model. For MyDT+, minimum leaf size $m = 4$ was selected because it provided a better balance between recall and F_1 -score for the died class. Hyperparameter sweep results of MyKNN, MyKNN+, and DT+ are listed in Appendix section 8.1.

Classifier	Setting	Accuracy	Precision (died)	Recall (died)	F -measure / F_1 (died)
ZeroR	Weka	67.03%	N/A	0.000	N/A
OneR	Weka	71.79%	0.582	0.511	0.544
Weka kNN / IBk	$k = 1$	69.23%	0.532	0.556	0.543
Naive Bayes	Weka	71.06%	0.585	0.422	0.490
MLP	Weka	63.74%	0.446	0.411	0.428
SVM / SMO	Weka	72.53%	0.595	0.522	0.556
MyKNN	$k = 3$	53.04%	0.357	0.522	0.423
MyKNN+	$k = 7$	70.34%	0.573	0.422	0.470
DT unpruned / J48	Weka	65.93%	0.484	0.489	0.486
DT pruned / J48	Weka	72.16%	0.635	0.367	0.465
MyDT	Standard DT	63.77%	0.466	0.522	0.486
MyDT+	$m = 4$	65.93%	0.487	0.467	0.470
Bagging with J48	Weka	69.60%	0.569	0.322	0.411
Boosting with J48	Weka	67.40%	0.506	0.478	0.491
Random Forest	Weka	68.13%	0.524	0.367	0.431

Table 1: Comparison of classifier performance on the heart dataset using 10-fold cross-validation.

4.2 Discussion

4.2.1 Overall classifier performance

Table 1 compares the performance of all classifiers using accuracy, precision, recall and F_1 -score for the “died” class. Among all classifiers, the SVM/SMO model achieved the best overall performance, obtaining the highest accuracy of 72.53% and the highest F_1 -score of 0.556. This indicates that SVM provided the most balanced classification performance between precision and recall. Weka’s pruned J48 Decision Tree also achieved strong performance with the accuracy of 72.16% and the highest precision for the “died” class (0.635), meaning that when the model predicted a patient would die, it was more likely to be correct. However, its recall was relatively low (0.367), showing that many actual death cases were missed.

Weka’s IBK classifier achieved the highest recall value (0.556) meaning it identified the largest number of actual death cases among all classifiers. It also achieved a strong

F_1 -score of 0.543, demonstrating balanced classification performance. In comparison, the custom classifiers generally achieved lower performance than Weka’s implementation, although some models still produced competitive results. MyKNN+ achieved an accuracy of 70.34%, which was higher than the MyKNN classifier (53.04%) and comparable to several Weka classifiers such as bagging with J48 and IBK. This suggests that the modifications made in KNN+ improved the classifier significantly.

The ensemble methods, including Bagging and Random Forest, did not perform as strongly as expected on this dataset. Their recall values for the “died” class were relatively low, suggesting that these models were more biased towards predicting the majority “survived” class. ZeroR achieved a relatively high accuracy of 67.03% despite having zero recall for the “died” class because it simply predicts the majority class in the dataset (survived) for every instance. As a result, ZeroR is not suitable for this classification task since it completely fails to identify patients who died. This demonstrates that accuracy alone is not sufficient for evaluating imbalance classification problems, and additional measures such as recall and F_1 -score are necessary.

Overall, the results show that different classifiers prioritise different aspects of performance, and evaluation metrics such as precision, recall and F_1 -score are important for understanding the strengths and weaknesses of each model.

4.2.2 Effect of modified algorithms (Comparison of MyKNN with MyKNN+ and MyDT with MyDT+)

The modified classifiers generally performed better than the original implementations. MyKNN+ achieved a significant improvement over the original MyKNN classifier. The accuracy increased from 53.04% to 70.34%, while precision for the “died” class improved from 0.357 to 0.573. The F_1 -score also increased from 0.423 to 0.470, showing that MyKNN+ produced a more balanced classification performance. These improvements were mainly due to the introduction of feature-weighted distance calculation and weighted voting. Important medical features contributed more strongly to the distance calculation, while closer neighbours had greater influence during prediction. As a result, MyKNN+ was able to make more accurate and reliable classifications than the standard KNN model.

Similarly, MyDT+ slightly improved the performance of the original MyDT classifier. The accuracy increased from 63.77% to 65.93%, and precision improved from 0.466 to 0.487. This improvement was achieved by introducing a minimum leaf size restriction, which prevented the tree from creating very small branches and reduced overfitting. However, the recall for the “died” class decreased from 0.522 to 0.467, indicating that MyDT+ became more conservative when predicting death cases. Although fewer false positives were produced, more actual death cases were missed. Overall, both modified classifiers successfully improved the baseline models, with MyKNN+ showing the largest overall performance improvement.

4.2.3 Comparison with Weka

The performance of the custom KNN and Decision Tree classifiers was compared with Weka's IBk and J48 implementations using 10-fold cross-validation. Overall, Weka's classifiers achieved higher accuracy than the custom implementations. Weka's IBk classifier achieved an accuracy of 69.23% and an F_1 -score of 0.543 for the "died" class, while the custom KNN classifier achieved an accuracy of 53.04% and an F_1 -score of 0.423. Weka's KNN also produced higher recall for the "died" class, meaning it identified more death cases correctly. One reason for the better performance is that Weka uses Euclidean distance calculations and additional optimisations for neighbour searching, while the custom KNN implementation uses a simpler categorical distance calculation based on feature mismatches. In addition, Weka's implementation is highly optimised and designed to handle classification tasks more efficiently.

Similarly, Weka's J48 classifier outperformed the custom Decision Tree classifier in overall accuracy, achieving 72.16% compared with 63.77% from the custom DT model. However, the custom Decision Tree achieved higher recall for the "died" class (0.522 compared with 0.367 in Weka J48), meaning it detected more actual death cases. On the other hand, Weka's J48 achieved higher precision and overall accuracy because it produced fewer false positive predictions. The better performance of J48 is likely due to its advanced pruning methods and more sophisticated tree optimisation techniques, which reduce overfitting and improve generalisation. In comparison, the custom DT implementation used a simpler information gain approach and did not include the same level of pruning and optimisation as Weka's implementation. Overall, although Weka's classifiers achieved stronger overall performance, the custom implementations still produced reasonable results and demonstrated that the core machine learning algorithms were implemented successfully.

4.2.4 Effect of pruning in Weka J48 (DT-unpruned and DT-pruned)

Weka's pruned and unpruned J48 Decision Tree classifiers were compared to examine the effect of pruning. The pruned J48 achieved a higher overall accuracy of 72.16%, compared with 65.93% for the unpruned J48. It also achieved better weighted precision, recall and F_1 -score(F-measure), with a weighted F_1 -score of 0.697 compared with 0.660 for the unpruned tree. This suggests that pruning improved the overall predictive performance of the Decision Tree on this dataset.

The difference in tree complexity was also substantial. The pruned tree had only 11 leaves and a tree size of 17, while the unpruned tree had 79 leaves and a tree size of 124. This shows that the unpruned tree created many more branches and was much more complex. Although the unpruned tree had a higher recall for the died class (0.489 compared with 0.367 for the pruned tree), its lower overall accuracy and F_1 -score suggest that it likely overfitted the training data. In contrast, the pruned tree removed overly specific branches and produced a simpler model that generalised better during 10 fold cross validation. Therefore, pruning had a positive effect overall by reducing model complexity and improving generalisation.

Measure	J48 Pruned	J48 Unpruned
Accuracy	72.16%	65.93%
Correctly Classified	197 / 273	180 / 273
Weighted Precision	0.707	0.660
Weighted Recall	0.722	0.659
Weighted F_1	0.697	0.660
Died Recall	0.367	0.489
Survived Recall	0.896	0.743
Number of Leaves	11	79
Tree Size	17	124

Table 2: Comparison of pruned and unpruned J48 Decision Tree performance on the heart dataset

4.2.5 Comparison of Weka tree based classifiers -single tree and ensemble of trees (DT-pruned, Bagging, Boosting and Random Forest)

Among Weka’s tree based classifiers, the single pruned J48 Decision Tree achieved the best overall predictive performance. It obtained an accuracy of 72.16%, with a weighted precision of 0.707, weighted recall of 0.722, and weighted F_1 -score (F-measure) of 0.697. In comparison, Bagging with J48 achieved 69.60% accuracy and a weighted F_1 -score of 0.669, Boosting with J48 achieved 67.40% accuracy and a weighted F_1 -score of 0.672, while Random Forest achieved 68.13% accuracy and a weighted F_1 -score of 0.664.

This result suggests that, for this dataset, the single pruned J48 tree generalised better overall than the ensemble tree based methods. Although ensemble methods such as Bagging, Boosting and Random Forest often improve stability by combining multiple trees, they did not outperform the pruned J48 classifier in this case. One possible reason is that the dataset is relatively small, with only 273 instances, and the attributes are already discretised into categorical values. Therefore, the main predictive patterns may already be captured by a simpler pruned tree, leaving limited additional benefit for more complex ensemble methods.

However, Boosting had the highest recall for the died class, with a recall of 0.478, compared with 0.367 for both pruned J48 and Random Forest, and 0.322 for Bagging. This means Boosting identified more of the patients who actually died, which may be important in a medical prediction context where missing high risk patients is costly. However, this came with lower overall accuracy and lower recall for the survived class. Overall, the pruned J48 model provided the best balance of accuracy and weighted F_1 -score, while Boosting may be more useful if the priority is detecting more died cases.

Tree-based Classifier	Accuracy	Weighted Precision	Weighted Recall	Weighted F_1	Died Recall	Survived Recall
DT pruned/J48	72.16%	0.707	0.722	0.697	0.367	0.896
Bagging with J48	69.60%	0.674	0.696	0.669	0.322	0.880
Boosting with J48	67.40%	0.670	0.674	0.672	0.478	0.770
Random Forest	68.13%	0.661	0.681	0.664	0.367	0.836

Table 3: Weka tree-based classifiers comparison.

5 Conclusion

This project investigated the performance of multiple machine learning classifiers for predicting patient survival on the heart failure dataset. The result showed that different classifiers performed differently depending on the evaluation measured used. Among all classifiers, Weka’s SVM and J48 models achieved the strongest overall performance, while the custom implementation also produced reasonable and competitive results. The modified classifiers, MyKNN+ and MyDT+, generally improved upon the MyKNN and MyDT algorithms. In particular, MyKNN+ achieved a significant increase in accuracy and F_1 -score through the use of feature-weighted distance calculation and weighted voting. The result also demonstrated that accuracy alone is not sufficient for evaluating classification performance on imbalance datasets. For example, ZeroR achieved relatively high accuracy despite completely failing to identify death cases. Therefore, additional evaluation measures such as precision, recall and F_1 -score are necessary for understanding the true performance of a classifier. Overall, the project demonstrated that machine learning methods can be effectively applied to heart disease survival prediction and that modifications to baseline algorithms can improve classification performance.

Several improvements could be explored in future work to further improve classification performance. First, additional feature selection or feature engineering techniques could be applied to better identify the most important medical attributes for prediction. Second, more advanced hyperparameter tuning methods could be used to optimise parameters such as the value of k in KNN or pruning settings in Decision Trees. Future work could also investigate more sophisticated ensemble learning methods or deep learning approaches to improve predictive performance. Moreover, handling class imbalance more effectively through resampling methods or cost-sensitive learning may improve the detection of death cases. Increasing the size of the dataset would also likely improve model performance by allowing the classifiers to learn more representative patterns and reduce overfitting caused by limited training data. In conclusion, testing the classifiers on larger and more diverse medical datasets would provide a more reliable evaluation of their generalisation ability and practical usefulness in real-world healthcare application.

6 Reflection

SID: 540493238

Through this assignment, the most important thing I learned was that classifier performance cannot be judged by accuracy alone. At first, I thought the model with the highest accuracy would automatically be the best model. However, after comparing precision, recall, F_1 -score, and discussing with my group members, I realised that this is not always true, especially for an imbalance dataset like the heart failure dataset.

I also learned how different classifiers make different trade offs. KNN and Decision Trees are based on very different ideas: KNN uses similarity between examples, while DT uses attribute splits to form rules. Through implementing and modifying these algorithms, I understood how small design choices, such as choosing k , using weighted voting, or adding a minimum leaf size, can affect the balance between accuracy and recall. Overall, this assignment helped me better understand not only how machine learning classifiers work, but also how to interpret their results critically in the context of a real prediction problem.

SID: 530524801

Through this project, I gained a better understanding of how machine learning classifiers work and how different evaluation measures can affect the interpretation of model performance. Before completing this assignment, I mainly focused on accuracy when comparing classifiers. However, I learned that for imbalance datasets, measures such as precision, recall and F_1 -score are also very important. For example, ZeroR achieved relatively high accuracy even though it completely failed to predict any death cases. This helps me understand why relying only on accuracy can be misleading.

I also learned how small modifications to algorithms can improve performance. By developing MyKNN+ and MyDT+, I gained practical experience in improving machine learning models through feature weighting and pruning techniques. In addition, implementing the classifiers from scratch helps me better understand the internal logic KNN and Decision Trees rather than only using existing libraries. Overall, this project improved both my programming skills and my understanding of machine learning concepts, especially in model evaluation, algorithm design and performance analysis.

SID: 520523553

Through this assignment, the most important thing I learned was how strongly machine learning results depend on evaluation design and problem context, not just on implementing a model correctly. I already had some prior experience with machine learning, so the core ideas of KNN, decision trees, and classification were familiar to me. However, implementing these algorithms from scratch made me think more carefully about details that are often hidden by libraries like scikit-learn or TensorFlow, such as tie-breaking rules, stopping conditions, and how small design choices affect model behaviour in practice.

I also found it valuable to analyse the trade-offs between different evaluation metrics in a more meaningful way. Because the dataset is class-imbalanced and the “died” class is the more important outcome, this assignment reinforced that accuracy alone is not enough. Some models achieved better overall accuracy but weaker recall for “died”, while others were more sensitive to the minority class at the cost of overall performance. This helped me understand machine learning more as a process of balancing objectives, interpreting results carefully, and making decisions based on the task rather than on a single number.

7 Acknowledgement of AI use

SID: 540493238

I used ChatGPT to support the completion of part 1 and part 2 of this assignment. For the coding component, I used ChatGPT to help explain machine learning concepts, how to approach and implement the questions. I also used it to help understand error messages and clarify how to interpret the classifier results. The final code was reviewed and edited by the group to ensure it matched the assignment requirements.

For the report component, I used ChatGPT to help reword parts of my writing, improve grammar, and make the explanations clearer and more structured. I also used it to help interpret the results tables, suggestion to make decisions and explain concepts in context of the heart failure prediction task(e.g. For accuracy, precision, recall and F_1 -score, what score is more important to consider in this task). The final report content, analysis and conclusion were written, edited together by the group before submission.

SID: 530524801

AI tools , ChatGPT, were used during this project to assist with understanding machine learning concepts, debugging programming errors, improving code structure and explaining algorithm behaviour during the implementation of the classifiers. The AI tools were also used to help identify issues in the code and suggest possible improvements. All final code implementations and modifications were reviewed, tested and completed by us.

Moreover, AI was also used to assist with improving grammar, sentence structure and clarity in the report writing process. The AI tools were also used to help explain evaluation results and organise discussion sections more clearly. All analysis, interpretations and final written content were reviewed and edited by me.

SID: 520523553

I used ChatGPT during both Part 1 and Part 2 of this assignment. For Part 1, I used it to help explain machine learning concepts, mathematically validating implementation ideas, debug code, and better understand error messages. All final code was written, reviewed, and tested by our group.

For Part 2, I used ChatGPT to improve grammar, clarity, and structure in the report. All final analysis and written content were reviewed and edited by us.

8 Appendix

8.1 Hyperparameter Sweeps

k	Accuracy	Precision (died)	Recall (died)	F_1 (died)
1	0.4316	0.3256	0.6889	0.4407
3	0.5304	0.3570	0.5222	0.4227
5	0.6152	0.4198	0.4889	0.4486
7	0.6336	0.4366	0.4222	0.4198
9	0.6513	0.4713	0.4000	0.4278

Table 4: Cross-validation results for MyKNN with different values of k .

k	Accuracy	Precision (died)	Recall (died)	F_1 (died)
1	0.6003	0.4194	0.4889	0.4473
3	0.6630	0.4935	0.5000	0.4939
5	0.6595	0.4739	0.4111	0.4370
7	0.7034	0.5733	0.4222	0.4700
9	0.6888	0.5471	0.3667	0.4287

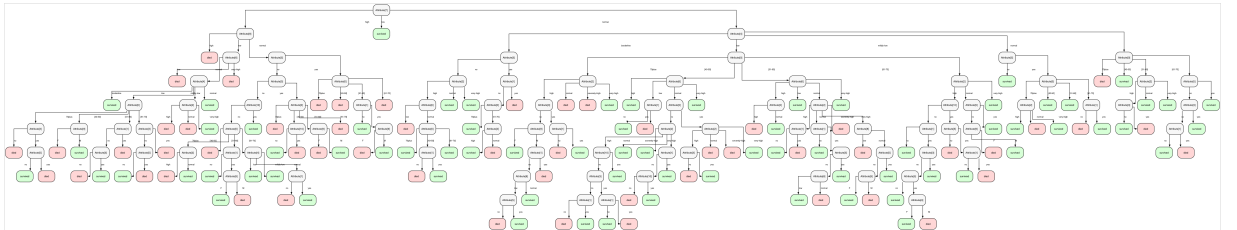
Table 5: Cross-validation results for MyKNN+ with different values of k .

Min leaf size	Accuracy	Precision (died)	Recall (died)	F_1 (died)
2	0.6050	0.4199	0.5000	0.4506
3	0.6266	0.4137	0.3889	0.3892
4	0.6593	0.4871	0.4667	0.4698
5	0.6704	0.4966	0.4222	0.4520

Table 6: Cross-validation results for MyDT+ with different values of minimum leaf size.

8.2 Diagrams of Decision Trees

DT



```

=== MyDT (whole dataset) ===
serum_creatinine = high
|   serum_sodium = high: died
|   serum_sodium = low
|   |   platelets = low: died
|   |   platelets = normal
|   |   |   ejection_fraction = borderline: survived
|   |   |   ejection_fraction = low
|   |   |   |   age = 70plus
|   |   |   |   |   diabetes = no: died
|   |   |   |   |   diabetes = yes
|   |   |   |   |   |   high_blood_pressure = no: survived
|   |   |   |   |   |   high_blood_pressure = yes: died
|   |   |   |   age = [40-50]
|   |   |   |   |   sex = F: died
|   |   |   |   |   sex = M: survived
|   |   |   |   age = [51-60]
|   |   |   |   |   anaemia = no
|   |   |   |   |   |   diabetes = no: died
|   |   |   |   |   |   diabetes = yes: survived
|   |   |   |   |   anaemia = yes: died
|   |   |   |   age = [61-70]
|   |   |   |   |   diabetes = no
|   |   |   |   |   |   high_blood_pressure = no: survived
|   |   |   |   |   |   high_blood_pressure = yes: died
|   |   |   |   |   diabetes = yes: died
|   |   |   |   ejection_fraction = mildly-low
|   |   |   |   CPK = high: died
|   |   |   |   CPK = normal: died
|   |   |   |   CPK = very-high: survived
|   |   |   |   ejection_fraction = normal: survived
|   |   |   platelets = very-high: died
|   serum_sodium = normal
|   |   high_blood_pressure = no
|   |   |   diabetes = no
|   |   |   |   smoking = no
|   |   |   |   |   age = 70plus
|   |   |   |   |   |   CPK = high: died
|   |   |   |   |   |   CPK = normal: survived
|   |   |   |   |   age = [40-50]: died
|   |   |   |   |   age = [51-60]
|   |   |   |   |   |   anaemia = no: survived
|   |   |   |   |   |   anaemia = yes
|   |   |   |   |   |   |   sex = F: survived
|   |   |   |   |   |   |   sex = M: died
|   |   |   |   |   age = [61-70]
|   |   |   |   |   |   ejection_fraction = low: survived
|   |   |   |   |   |   ejection_fraction = mildly-low: survived
|   |   |   |   |   |   ejection_fraction = normal
|   |   |   |   |   |   |   anaemia = no: died
|   |   |   |   |   |   |   anaemia = yes: survived
|   |   |   |   |   smoking = yes: survived
|   |   |   |   diabetes = yes
|   |   |   |   |   age = 70plus: died
|   |   |   |   |   age = [40-50]
|   |   |   |   |   |   smoking = no: survived
|   |   |   |   |   |   smoking = yes: died
|   |   |   |   |   age = [51-60]
|   |   |   |   |   |   sex = F: died
|   |   |   |   |   |   sex = M: survived
|   |   |   |   |   age = [61-70]: died
|   |   |   high_blood_pressure = yes
|   |   |   |   age = 70plus: died
|   |   |   |   age = [40-50]: died
|   |   |   |   age = [51-60]
|   |   |   |   |   anaemia = no: survived
|   |   |   |   |   anaemia = yes
|   |   |   |   |   |   sex = F: died
|   |   |   |   |   |   sex = M: survived
|   |   |   |   |   age = [61-70]: died
|   serum_creatinine = low: survived
|   serum_creatinine = normal
|   |   ejection_fraction = borderline
|   |   |   diabetes = no
|   |   |   |   CPK = high
|   |   |   |   |   platelets = low: died
|   |   |   |   |   platelets = normal
|   |   |   |   |   |   age = 70plus: survived
|   |   |   |   |   |   age = [40-50]
|   |   |   |   |   |   |   anaemia = no: died
|   |   |   |   |   |   |   anaemia = yes: survived
|   |   |   |   |   |   age = [61-70]: survived
|   |   |   |   |   CPK = normal: survived
|   |   |   |   |   CPK = very-high: survived
|   |   |   |   diabetes = yes
|   |   |   |   |   anaemia = no
|   |   |   |   |   |   age = 70plus: survived
|   |   |   |   |   |   age = [61-70]
|   |   |   |   |   |   |   CPK = high: survived
|   |   |   |   |   |   |   CPK = normal: died
|   |   |   |   |   anaemia = yes: died
|   |   ejection_fraction = low
|   |   |   age = 70plus
|   |   |   |   CPK = high
|   |   |   |   |   diabetes = no
|   |   |   |   |   |   sex = F: survived
|   |   |   |   |   |   sex = M
|   |   |   |   |   |   |   anaemia = no
|   |   |   |   |   |   |   |   serum_sodium = low

```

```

| | | | | | | | high_blood_pressure = no: died
| | | | | | | | high_blood_pressure = yes: survived
| | | | | | | | serum_sodium = normal: survived
| | | | | | | | anaemia = yes: died
| | | | | | | | diabetes = yes
| | | | | | | | | anaemia = no: died
| | | | | | | | | anaemia = yes: survived
| | | | | | | | CPK = normal: died
| | | | | | | | CPK = severely-high: died
| | | | | | | | CPK = very-high: survived
| | | | | | | | age = [40-50]
| | | | | | | | | platelets = high: survived
| | | | | | | | | platelets = low
| | | | | | | | | | anaemia = no: survived
| | | | | | | | | | anaemia = yes: died
| | | | | | | | | platelets = normal
| | | | | | | | | | high_blood_pressure = no
| | | | | | | | | | | CPK = high
| | | | | | | | | | | | smoking = no
| | | | | | | | | | | | | diabetes = no
| | | | | | | | | | | | | | anaemia = no: died
| | | | | | | | | | | | | | anaemia = yes: survived
| | | | | | | | | | | | | | diabetes = yes
| | | | | | | | | | | | | | | anaemia = no: survived
| | | | | | | | | | | | | | | anaemia = yes: died
| | | | | | | | | | | | | | | smoking = yes: survived
| | | | | | | | | | | | | | CPK = normal: survived
| | | | | | | | | | | | | | CPK = severely-high: survived
| | | | | | | | | | | | | | CPK = very-high
| | | | | | | | | | | | | | | anaemia = no
| | | | | | | | | | | | | | | | smoking = no: died
| | | | | | | | | | | | | | | | smoking = yes: survived
| | | | | | | | | | | | | | | anaemia = yes: survived
| | | | | | | | | | | | | | high_blood_pressure = yes
| | | | | | | | | | | | | | | CPK = high
| | | | | | | | | | | | | | | | diabetes = no: died
| | | | | | | | | | | | | | | | diabetes = yes: survived
| | | | | | | | | | | | | | | CPK = normal: died
| | | | | | | | | | | | | | | CPK = severely-high: died
| | | | | | | | | | | | | | | CPK = very-high: survived
| | | | | | | | | | | | | | platelets = very-high: survived
| | | | | | | | | | | | | | age = [51-60]
| | | | | | | | | | | | | | | platelets = high
| | | | | | | | | | | | | | | | CPK = high: died
| | | | | | | | | | | | | | | | CPK = normal: survived
| | | | | | | | | | | | | | | platelets = low: survived
| | | | | | | | | | | | | | | platelets = normal
| | | | | | | | | | | | | | | | CPK = high
| | | | | | | | | | | | | | | | | anaemia = no: survived
| | | | | | | | | | | | | | | | | anaemia = yes: died
| | | | | | | | | | | | | | | | CPK = normal
| | | | | | | | | | | | | | | | | anaemia = no: survived
| | | | | | | | | | | | | | | | | anaemia = yes
| | | | | | | | | | | | | | | | | | high_blood_pressure = no
| | | | | | | | | | | | | | | | | | | serum_sodium = low: survived
| | | | | | | | | | | | | | | | | | | serum_sodium = normal: died
| | | | | | | | | | | | | | | | | | | high_blood_pressure = yes: survived
| | | | | | | | | | | | | | | | | CPK = severely-high: died
| | | | | | | | | | | | | | | | | CPK = very-high
| | | | | | | | | | | | | | | | | | serum_sodium = low: died
| | | | | | | | | | | | | | | | | | serum_sodium = normal
| | | | | | | | | | | | | | | | | | | high_blood_pressure = no
| | | | | | | | | | | | | | | | | | | sex = F: survived
| | | | | | | | | | | | | | | | | | | sex = M: died
| | | | | | | | | | | | | | | | | | high_blood_pressure = yes: survived
| | | | | | | | | | | | | | | | | platelets = very-high: survived
| | | | | | | | | | | | | | | | age = [61-70]
| | | | | | | | | | | | | | | | | CPK = high
| | | | | | | | | | | | | | | | | | smoking = no
| | | | | | | | | | | | | | | | | | | anaemia = no: survived
| | | | | | | | | | | | | | | | | | | anaemia = yes
| | | | | | | | | | | | | | | | | | | | diabetes = no
| | | | | | | | | | | | | | | | | | | | | high_blood_pressure = no: survived
| | | | | | | | | | | | | | | | | | | | | high_blood_pressure = yes
| | | | | | | | | | | | | | | | | | | | | sex = F: survived
| | | | | | | | | | | | | | | | | | | | | sex = M: died
| | | | | | | | | | | | | | | | | | | | diabetes = yes: survived
| | | | | | | | | | | | | | | | | | smoking = yes
| | | | | | | | | | | | | | | | | | | high_blood_pressure = no: died
| | | | | | | | | | | | | | | | | | | high_blood_pressure = yes
| | | | | | | | | | | | | | | | | | | | anaemia = no: survived
| | | | | | | | | | | | | | | | | | | | anaemia = yes: died
| | | | | | | | | | | | | | | | | CPK = normal
| | | | | | | | | | | | | | | | | | high_blood_pressure = no
| | | | | | | | | | | | | | | | | | | sex = F: died
| | | | | | | | | | | | | | | | | | | sex = M: survived
| | | | | | | | | | | | | | | | | | | high_blood_pressure = yes: survived
| | | | | | | | | | | | | | | | | | CPK = very-high: survived
| | | | | | | | | | | | | | | | | ejection_fraction = mildly-low
| | | | | | | | | | | | | | | | | | high_blood_pressure = no: survived
| | | | | | | | | | | | | | | | | | high_blood_pressure = yes
| | | | | | | | | | | | | | | | | | | age = 70plus
| | | | | | | | | | | | | | | | | | | CPK = high: survived
| | | | | | | | | | | | | | | | | | | CPK = normal: died
| | | | | | | | | | | | | | | | | | | CPK = very-high: survived
| | | | | | | | | | | | | | | | | | | age = [40-50]: survived
| | | | | | | | | | | | | | | | | | | age = [51-60]: survived
| | | | | | | | | | | | | | | | | | | age = [61-70]
| | | | | | | | | | | | | | | | | | | anaemia = no: died

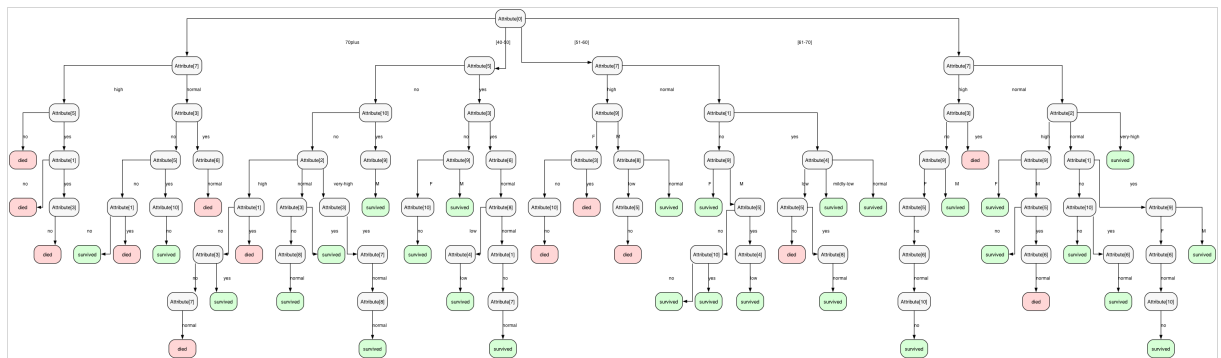
```

```

|   |   |   |   anaemia = yes: survived
|   |   |   |   ejection_fraction = normal
|   |   |   |   |   age = 70plus: died
|   |   |   |   |   age = [40-50]: survived
|   |   |   |   |   age = [51-60]
|   |   |   |   |   |   CPK = high
|   |   |   |   |   |   |   diabetes = no: survived
|   |   |   |   |   |   |   diabetes = yes: died
|   |   |   |   |   |   |   CPK = normal: survived
|   |   |   |   |   |   |   CPK = very-high: survived
|   |   |   |   |   |   age = [61-70]
|   |   |   |   |   |   |   high_blood_pressure = no
|   |   |   |   |   |   |   |   diabetes = no
|   |   |   |   |   |   |   |   |   anaemia = no: survived
|   |   |   |   |   |   |   |   |   anaemia = yes: died
|   |   |   |   |   |   |   |   |   diabetes = yes: survived
|   |   |   |   |   |   |   |   |   high_blood_pressure = yes: survived

```

DT+



```

=== MyDT+ (whole dataset) ===
age = 70plus
|   serum_creatinine = high
|   |   high_blood_pressure = no: died
|   |   high_blood_pressure = yes
|   |   |   anaemia = no: died
|   |   |   anaemia = yes
|   |   |   |   diabetes = no: died
|   serum_creatinine = normal
|   |   diabetes = no
|   |   |   high_blood_pressure = no
|   |   |   |   anaemia = no: survived
|   |   |   |   anaemia = yes: died
|   |   |   |   high_blood_pressure = yes
|   |   |   |   |   smoking = no: survived
|   |   |   |   diabetes = yes
|   |   |   |   |   platelets = normal: died
age = [40-50]
|   high_blood_pressure = no
|   |   smoking = no
|   |   |   CPK = high
|   |   |   |   anaemia = no
|   |   |   |   |   diabetes = no
|   |   |   |   |   |   serum_creatinine = normal: died
|   |   |   |   |   |   diabetes = yes: survived
|   |   |   |   |   |   anaemia = yes: died
|   |   |   |   CPK = normal
|   |   |   |   |   diabetes = no
|   |   |   |   |   |   serum_sodium = normal: survived
|   |   |   |   |   |   diabetes = yes: survived
|   |   |   |   CPK = very-high
|   |   |   |   |   diabetes = yes
|   |   |   |   |   |   serum_creatinine = normal
|   |   |   |   |   |   |   serum_sodium = normal: survived
|   |   smoking = yes
|   |   |   sex = M: survived
|   high_blood_pressure = yes
|   |   diabetes = no
|   |   |   sex = F
|   |   |   |   smoking = no: survived
|   |   |   |   sex = M: survived
|   |   diabetes = yes
|   |   |   platelets = normal
|   |   |   |   serum_sodium = low
|   |   |   |   |   ejection_fraction = low: survived
|   |   |   |   serum_sodium = normal
|   |   |   |   |   anaemia = no

```



```

| | | | | serum_creatinine = normal: survived
age = [51-60]
| | serum_creatinine = high
| | | sex = F
| | | | diabetes = no
| | | | | smoking = no: died
| | | | diabetes = yes: died
| | | sex = M
| | | | serum_sodium = low
| | | | | high_blood_pressure = no: died
| | | | serum_sodium = normal: survived
| | serum_creatinine = normal
| | | anaemia = no
| | | | sex = F: survived
| | | | sex = M
| | | | | high_blood_pressure = no
| | | | | | smoking = no: survived
| | | | | | smoking = yes: survived
| | | | | high_blood_pressure = yes
| | | | | | ejection_fraction = low: survived
| | | anaemia = yes
| | | | ejection_fraction = low
| | | | | high_blood_pressure = no: died
| | | | | high_blood_pressure = yes
| | | | | | serum_sodium = normal: survived
| | | | ejection_fraction = mildly-low: survived
| | | ejection_fraction = normal: survived
age = [61-70]
| | serum_creatinine = high
| | | diabetes = no
| | | | sex = F
| | | | | high_blood_pressure = no
| | | | | | platelets = normal
| | | | | | smoking = no: survived
| | | | sex = M: survived
| | | | diabetes = yes: died
| | | serum_creatinine = normal
| | | | CPK = high
| | | | | sex = F: survived
| | | | | sex = M
| | | | | | high_blood_pressure = no: survived
| | | | | | high_blood_pressure = yes
| | | | | | platelets = normal: died
| | | | CPK = normal
| | | | | anaemia = no
| | | | | | smoking = no: survived
| | | | | | smoking = yes
| | | | | | | platelets = normal: survived
| | | | | anaemia = yes
| | | | | | sex = F
| | | | | | | platelets = normal
| | | | | | | smoking = no: survived
| | | | | | sex = M: survived
| | | CPK = very-high: survived

```

8.3 Python Implementation of KNN+ and DT+

This appendix contains the Python implementation of our modified classifiers, MyKNN+ and MyDT+. The code also includes the shared dataset and baseline classifier structure required to support these implementations.

MyKNN+ extends the standard K-Nearest Neighbour classifier by introducing feature-weighted distance and distance-weighted voting, so that more informative attributes and closer neighbours have greater influence on the final prediction. MyDT+ extends the standard decision tree by enforcing a minimum leaf size constraint, which prevents splits that would create very small child nodes and therefore reduces overly specific branches.

The implementation below is included for completeness and reproducibility. It reflects the final versions used in our experiments.

```

1 from __future__ import annotations
2 import math
3 from abc import ABC, abstractmethod
4 from collections import Counter
5 from typing import Any

```

```

6  import numpy as np
7
8  # =====
9  # Dataset
10 # =====
11 class Dataset:
12     # X stores the feature matrix, y stores the label vector (or None for test data).
13     X: np.ndarray
14     y: np.ndarray | None
15
16     def __init__(self, X: np.ndarray, y: np.ndarray | None=None):
17         # Basic shape checks to keep the dataset representation consistent.
18         if X.ndim != 2:
19             raise ValueError("X must be a 2D array")
20         if y is not None and y.ndim != 1:
21             raise ValueError("y must be a 1D array")
22         if y is not None and X.shape[0] != y.shape[0]:
23             raise ValueError("X and y must have the same number of rows")
24
25         self.X = X
26         self.y = y
27
28     def __len__(self) -> int:
29         # Number of examples in the dataset.
30         return int(self.X.shape[0])
31
32     @property
33     def n_features(self) -> int:
34         # Number of attributes per example.
35         if self.X.ndim != 2:
36             return 0
37         return int(self.X.shape[1])
38
39     @property
40     def has_labels(self) -> bool:
41         # True for training data, False for unlabeled testing data.
42         return self.y is not None
43
44     @classmethod
45     def from_training_file(cls, filename: str) -> Dataset:
46         # Read a labelled csv file of the form:
47         # feature_1,...,feature_d,class_label
48         rows: list[list[str]] = []
49         labels: list[str] = []
50
51         with open(filename, "r", encoding="utf-8") as f:
52             for line_num, line in enumerate(f, start=1):
53                 line = line.strip()
54                 if not line:
55                     continue
56
57                 parts = [part.strip() for part in line.split(",")]
58                 if len(parts) < 2:
59                     raise ValueError(
60                         f"Invalid training row at line {line_num}: expected at least 2 columns."
61                     )
62
63                 rows.append(parts[:-1])
64                 labels.append(parts[-1])
65
66         if not rows:

```

```

67         raise ValueError("Training file is empty.")
68
69     X = np.array(rows, dtype=object)
70     y = np.array(labels, dtype=object)
71
72     if X.ndim != 2:
73         raise ValueError("Training data must be a 2D table.")
74
75     return cls(X, y)
76
77     @classmethod
78     def from_testing_file(cls, filename: str) -> Dataset:
79         # Read an unlabeled csv file of the form:
80         # feature_1,...,feature_d
81         rows: list[list[str]] = []
82
83         with open(filename, "r", encoding="utf-8") as f:
84             for line_num, line in enumerate(f, start=1):
85                 line = line.strip()
86                 if not line:
87                     continue
88
89                 parts = [part.strip() for part in line.split(",")]
90                 if len(parts) < 1:
91                     raise ValueError(
92                         f"Invalid testing row at line {line_num}: expected at least 1 column."
93                     )
94
95                 rows.append(parts)
96
97         if not rows:
98             raise ValueError("Testing file is empty.")
99
100     X = np.array(rows, dtype=object)
101
102     if X.ndim != 2:
103         raise ValueError("Testing data must be a 2D table.")
104
105     return cls(X, None)
106
107     @classmethod
108     def from_rows(cls, rows: list[list[str]], has_labels: bool = True) -> Dataset:
109         # Convenience constructor used by evaluation code when folds are already in memory.
110         if not rows:
111             raise ValueError("Rows are empty.")
112
113         arr = np.array(rows, dtype=object)
114         if arr.ndim != 2:
115             raise ValueError("Rows must form a 2D table.")
116
117         if has_labels:
118             if arr.shape[1] < 2:
119                 raise ValueError("Labeled rows must contain at least one feature and one label.")
120             X = arr[:, :-1]
121             y = arr[:, -1]
122             return cls(X, y)
123
124         return cls(arr, None)
125
126     def rows_with_labels(self) -> list[list[str]]:
127         # Convert the dataset back into row form with labels appended at the end.

```

```

128         if self.y is None:
129             raise ValueError("This dataset has no labels.")
130         return [list(row) + [str(label)] for row, label in zip(self.X, self.y)]
131
132     def subset(self, indices: list[int]) -> Dataset:
133         # Return a smaller dataset containing only the selected rows.
134         X_sub = self.X[indices]
135         if self.y is None:
136             return Dataset(X_sub, None)
137         y_sub = self.y[indices]
138         return Dataset(X_sub, y_sub)
139
140     # =====
141     # Abstract Parent Classifier class
142     # =====
143     class BaseClassifier(ABC):
144         # Common interface for all classifiers in this assignment.
145
146         @abstractmethod
147         def fit(self, X: np.ndarray, y: np.ndarray) -> BaseClassifier:
148             raise NotImplementedError
149
150         @abstractmethod
151         def predict(self, X: np.ndarray) -> list[str]:
152             raise NotImplementedError
153
154         @abstractmethod
155         def predict_one(self, x: np.ndarray) -> str:
156             raise NotImplementedError
157
158         def _check_X_y(self, X: np.ndarray, y: np.ndarray) -> None:
159             # Shared training-data validation.
160             if X.size == 0 or y.size == 0:
161                 raise ValueError("X and y must be non-empty.")
162             if X.ndim != 2:
163                 raise ValueError("X must be a 2D array.")
164             if y.ndim != 1:
165                 raise ValueError("y must be a 1D array.")
166             if X.shape[0] != y.shape[0]:
167                 raise ValueError("X and y must have the same number of rows.")
168
169         def _check_X(self, X: np.ndarray) -> None:
170             # Shared testing-data validation.
171             if X.ndim != 2:
172                 raise ValueError("X must be a 2D array.")
173
174         def _check_is_fitted(self) -> None:
175             # Implemented by subclasses to ensure training has happened before prediction.
176             raise NotImplementedError
177
178     # =====
179     # KNeighbours Classifier class
180     # =====
181     class KNNClassifier(BaseClassifier):
182         # Standard KNN for nominal data.
183         # Distance ties are broken by smaller row index in the training file.
184         # Voting ties are broken in favour of "died".
185
186         def __init__(self, k: int=3) -> None:
187             self.k = k
188             self.X_train_: np.ndarray | None = None

```

```

189         self.y_train_: np.ndarray | None = None
190
191     def _check_is_fitted(self) -> None:
192         if self.X_train_ is None or self.y_train_ is None:
193             raise ValueError("KNNClassifier has not been fitted yet.")
194
195     def fit(self, X: np.ndarray, y: np.ndarray) -> KNNClassifier:
196         # Store the training set for instance-based prediction.
197         X = np.array(X, dtype=object)
198         y = np.array(y, dtype=object)
199
200         self._check_X_y(X, y)
201
202         if self.k <= 0:
203             raise ValueError("k must be a positive integer")
204         if self.k > X.shape[0]:
205             raise ValueError("k cannot be larger than the number of training examples")
206
207         self.X_train_ = X
208         self.y_train_ = y
209
210         return self
211
212     def _compute_distances(self, x: np.ndarray) -> np.ndarray:
213         # Euclidean distance for nominal attributes:
214         # each attribute contributes 0 if equal, 1 if different,
215         # so the total distance is sqrt(number of mismatches).
216         self._check_is_fitted()
217         assert self.X_train_ is not None
218
219         x = np.array(x, dtype=object)
220         if x.ndim != 1:
221             raise ValueError("A single test example must be a 1D array.")
222         if x.shape[0] != self.X_train_.shape[1]:
223             raise ValueError("Test example has a different number of features")
224
225         mismatches = np.sum(self.X_train_ != x, axis=1)
226         return np.sqrt(mismatches.astype(float))
227
228     def _get_k_nearest(self, x: np.ndarray) -> list[tuple[float, int, str]]:
229         # Return the k nearest neighbours as tuples:
230         # (distance, row_index, class_label)
231         self._check_is_fitted()
232         assert self.y_train_ is not None
233
234         distances = self._compute_distances(x)
235         indexed: list[tuple[float, int, str]] = [
236             (int(distances[i]), i, str(self.y_train_[i]))
237             for i in range(len(distances))
238         ]
239
240         # Tie-breaking:
241         # 1. smaller distance first
242         # 2. if equal distance, smaller training row index first
243         indexed.sort(key=lambda item: (item[0], item[1]))
244         return indexed[: self.k]
245
246     def _vote(self, neighbours: list[tuple[float, int, str]]) -> str:
247         # Majority vote among the k nearest neighbours.
248         # If there is a tie, predict "died".
249         died_count = sum(1 for _, _, label in neighbours if label == "died")

```

```

250         survived_count = sum(1 for _, _, label in neighbours if label == "survived")
251
252         return "died" if died_count >= survived_count else "survived"
253
254     def predict_one(self, x: np.ndarray) -> str:
255         # Predict the label of a single test example.
256         neighbours = self._get_k_nearest(x)
257         return self._vote(neighbours)
258
259     def predict(self, X: np.ndarray) -> list[str]:
260         # Predict labels for all rows in the test matrix.
261         self._check_is_fitted()
262
263         X = np.array(X, dtype=object)
264         self._check_X(X)
265
266         assert self.X_train_ is not None
267         if X.shape[1] != self.X_train_.shape[1]:
268             raise ValueError("Testing data must have the same number of features as training data.")
269
270         return [self.predict_one(row) for row in X]
271
272     # =====
273     # KNN+
274     # =====
275     class WeightedKNNClassifier(KNNClassifier):
276         # Modified KNN.
277         # This version learns feature weights from the training set and uses
278         # distance-weighted voting so closer neighbours have more influence.
279
280         def __init__(self, k: int = 7) -> None:
281             super().__init__(k)
282             self.feature_weights_: np.ndarray | None = None
283
284         def fit(self, X: np.ndarray, y: np.ndarray) -> "WeightedKNNClassifier":
285             # Fit the baseline KNN state, then estimate feature weights.
286             super().fit(X, y)
287
288             assert self.X_train_ is not None
289             assert self.y_train_ is not None
290
291             n_features = self.X_train_.shape[1]
292             weights = []
293
294             for feature_index in range(n_features):
295                 score = self._feature_importance(feature_index)
296                 weights.append(score)
297
298             weights_array = np.array(weights, dtype=float)
299
300             # Avoid the degenerate case where all feature scores are zero.
301             if np.sum(weights_array) == 0:
302                 weights_array = np.ones(n_features)
303
304             # Normalise weights so their average is approximately 1.
305             weights_array = weights_array / np.mean(weights_array)
306
307             self.feature_weights_ = weights_array
308             return self
309
310         def _feature_importance(self, feature_index: int) -> float:

```

```

311     # Estimate a simple feature importance score based on how strongly
312     # the feature values separate the two classes in the training set.
313     assert self.X_train_ is not None
314     assert self.y_train_ is not None
315
316     values = np.unique(self.X_train_[ :, feature_index])
317     total = len(self.y_train_)
318
319     score = 0.0
320
321     for value in values:
322         mask = self.X_train_[ :, feature_index] == value
323         subset_y = self.y_train_[mask]
324
325         died_count = np.sum(subset_y == "died")
326         survived_count = np.sum(subset_y == "survived")
327         subset_size = len(subset_y)
328
329         if subset_size == 0:
330             continue
331
332         died_rate = died_count / subset_size
333         survived_rate = survived_count / subset_size
334
335         # Larger class-rate difference means the feature is more informative.
336         score += (subset_size / total) * abs(died_rate - survived_rate)
337
338     return float(score)
339
340 def _compute_distances(self, x: np.ndarray) -> np.ndarray:
341     # Weighted nominal distance:
342     # mismatches on more informative features contribute more.
343     self._check_is_fitted()
344
345     assert self.X_train_ is not None
346     assert self.feature_weights_ is not None
347
348     x = np.array(x, dtype=object)
349
350     if x.ndim != 1:
351         raise ValueError("A single test example must be a 1D array.")
352
353     if x.shape[0] != self.X_train_.shape[1]:
354         raise ValueError("Test example has a different number of features.")
355
356     differences = self.X_train_ != x
357
358     # Important features contribute more to the total distance.
359     weighted_differences = differences * self.feature_weights_
360
361     return np.sum(weighted_differences, axis=1)
362
363 def _weighted_vote(self, neighbours: list[tuple[float, int, str]]) -> str:
364     # Distance-weighted vote:
365     # closer neighbours receive larger voting weight.
366     died_score = 0.0
367     survived_score = 0.0
368
369     for dist, index, label in neighbours:
370         weight = 1.0 / (dist + 1)
371

```

```

372         if label == "died":
373             died_score += weight
374         elif label == "survived":
375             survived_score += weight
376
377         return "died" if died_score >= survived_score else "survived"
378
379     def predict_one(self, x: np.ndarray) -> str:
380         # Predict the label of one test example using weighted voting.
381         neighbours = self._get_k_nearest(x)
382         return self._weighted_vote(neighbours)
383
384
385     # =====
386     # Decision Tree Classifier
387     # =====
388     class DecisionTreeNode:
389         # A single node in the decision tree.
390         # Internal nodes store an attribute index and outgoing children.
391         # Leaf nodes store a final class label.
392
393         def __init__(
394             self,
395             attribute_index: int | None=None,
396             label: str | None=None
397         ) -> None:
398             self.attribute_index = attribute_index
399             self.label = label
400             self.children: dict[str, DecisionTreeNode] = {}
401             self.majority_label: str | None = None
402
403         def is_leaf(self) -> bool:
404             return self.label is not None
405
406
407     class DecisionTreeClassifier(BaseClassifier):
408         # Standard decision tree for nominal attributes.
409         # Splits are chosen using Information Gain.
410         # No pruning is applied in the baseline tree.
411
412         def __init__(self):
413             self.root_: DecisionTreeNode | None = None
414             self.n_features_in_: int | None = None
415
416         def _check_is_fitted(self) -> None:
417             if self.root_ is None or self.n_features_in_ is None:
418                 raise ValueError("DecisionTreeClassifier has not been fitted yet.")
419
420         def fit(self, X: np.ndarray, y: np.ndarray) -> DecisionTreeClassifier:
421             # Build the full decision tree from the training set.
422             X = np.array(X, dtype=object)
423             y = np.array(y, dtype=object)
424
425             self._check_X_y(X, y)
426
427             self.n_features_in_ = int(X.shape[1])
428             attributes = list(range(self.n_features_in_))
429             self.root_ = self._build_tree(X, y, attributes)
430             return self
431
432         def predict(self, X: np.ndarray) -> list[str]:

```



```

433     # Predict labels for all rows in the test matrix.
434     self._check_is_fitted()
435
436     X = np.array(X, dtype=object)
437     self._check_X(X)
438
439     assert self.n_features_in_ is not None
440     if X.shape[1] != self.n_features_in_:
441         raise ValueError("Testing data must have the same number of features as training data.")
442
443     return [self.predict_one(row) for row in X]
444
445 def predict_one(self, x: np.ndarray) -> str:
446     # Traverse the tree from the root until a leaf is reached.
447     # If an unseen attribute value appears at test time, fall back to
448     # the majority class stored at the current node.
449     self._check_is_fitted()
450     assert self.root_ is not None
451     assert self.n_features_in_ is not None
452
453     x = np.array(x, dtype=object)
454     if x.ndim != 1:
455         raise ValueError("A single test example must be a 1D array.")
456     if x.shape[0] != self.n_features_in_:
457         raise ValueError("Test example has a different number of features.")
458
459     node = self.root_
460
461     while not node.is_leaf():
462         assert node.attribute_index is not None
463
464         value = str(x[node.attribute_index])
465
466         if value not in node.children:
467             assert node.majority_label is not None
468             return node.majority_label
469
470         node = node.children[value]
471
472     assert node.label is not None
473     return node.label
474
475 def _entropy(self, labels: np.ndarray) -> float:
476     # Entropy of the class distribution at a node.
477     if labels.size == 0:
478         raise ValueError("Cannot compute entropy of an empty label array.")
479
480     _, counts = np.unique(labels, return_counts=True)
481     probabilities = counts / counts.sum()
482
483     entropy = 0.0
484     for p in probabilities:
485         entropy -= float(p) * math.log2(float(p))
486
487     return entropy
488
489 def _majority_label(self, labels: np.ndarray) -> str:
490     # Return the majority class at the node.
491     # If there is a tie, choose "died" as required by the specification.
492     died_count = int(np.sum(labels == "died"))
493     survived_count = int(np.sum(labels == "survived"))

```

```

494
495     return "died" if died_count >= survived_count else "survived"
496
497 def _information_gain(self, X: np.ndarray, y: np.ndarray, attr_index: int) -> float:
498     # Compute Information Gain for splitting on one attribute.
499     base_entropy = self._entropy(y)
500     values = np.unique(X[:, attr_index])
501
502     remainder = 0.0
503     total = int(y.shape[0])
504
505     for value in values:
506         mask = X[:, attr_index] == value
507         subset_y = y[mask]
508         weight = subset_y.shape[0] / total
509         remainder += float(weight) * self._entropy(subset_y)
510
511     return base_entropy - remainder
512
513 def _partition(
514     self,
515     X: np.ndarray,
516     y: np.ndarray,
517     attr_index: int,
518 ) -> dict[str, tuple[np.ndarray, np.ndarray]]:
519     # Split the current dataset into subsets according to one attribute.
520     partitions: dict[str, tuple[np.ndarray, np.ndarray]] = {}
521     values = np.unique(X[:, attr_index])
522
523     for value in values:
524         value_str = str(value)
525         mask = X[:, attr_index] == value
526         partitions[value_str] = (X[mask], y[mask])
527
528     return partitions
529
530 def _build_tree(
531     self,
532     X: np.ndarray,
533     y: np.ndarray,
534     attributes: list[int],
535 ) -> DecisionTreeNode:
536     # Recursively construct the decision tree.
537     if y.size == 0:
538         raise ValueError("Cannot build a tree from an empty dataset.")
539
540     # Stopping condition 1: all labels at the node are the same.
541     if np.all(y == y[0]):
542         return DecisionTreeNode(label=str(y[0]))
543
544     # Stopping condition 2: no attributes left to split on.
545     if not attributes:
546         return DecisionTreeNode(label=self._majority_label(y))
547
548     # Select the attribute with maximum Information Gain.
549     best_attr = max(attributes, key=lambda a: self._information_gain(X, y, a))
550
551     node = DecisionTreeNode(attribute_index=best_attr)
552     node.majority_label = self._majority_label(y)
553
554     partitions = self._partition(X, y, best_attr)

```

```

555     remaining_attributes = [a for a in attributes if a != best_attr]
556
557     # Recursively build one child subtree for each attribute value.
558     for attr_value, (subset_X, subset_y) in partitions.items():
559         child = self._build_tree(subset_X, subset_y, remaining_attributes)
560         node.children[attr_value] = child
561
562     return node
563
564     def print_tree(self) -> None:
565         # Public helper for printing a simple text-based tree diagram.
566         self._check_is_fitted()
567         assert self.root_ is not None
568         self._print_tree(self.root_, depth=0)
569
570     def _print_tree(self, node: DecisionTreeNode, depth: int) -> None:
571         # Recursive printer used for the report appendix.
572         indent = " " * depth
573
574         if node.is_leaf():
575             assert node.label is not None
576             print(f"{indent}Leaf: {node.label}")
577             return
578
579         assert node.attribute_index is not None
580         print(f"{indent}Attribute[{node.attribute_index}]")
581
582         for value, child in node.children.items():
583             print(f"{indent} = {value}")
584             self._print_tree(child, depth + 1)
585
586
587     # =====
588     # Decision Tree Plus
589     # =====
590     class DecisionTreePlusClassifier(DecisionTreeClassifier):
591         # Modified decision tree.
592         # This version only allows splits that do not create very small child nodes.
593         # The goal is to reduce overly specific leaves and improve generalisation.
594
595         def __init__(self, min_samples_leaf: int = 3) -> None:
596             super().__init__()
597             self.min_samples_leaf = min_samples_leaf
598
599         def fit(self, X: np.ndarray, y: np.ndarray) -> DecisionTreePlusClassifier:
600             # Build the modified decision tree with a minimum leaf-size constraint.
601             X = np.array(X, dtype=object)
602             y = np.array(y, dtype=object)
603
604             self._check_X_y(X, y)
605
606             if self.min_samples_leaf < 1:
607                 raise ValueError("min_samples_leaf must be at least 1.")
608
609             self.n_features_in_ = int(X.shape[1])
610             attributes = list(range(self.n_features_in_))
611             self.root_ = self._build_tree(X, y, attributes)
612             return self
613
614         def _is_valid_split(
615             self,

```

```

616         X: np.ndarray,
617         y: np.ndarray,
618         attr_index: int,
619     ) -> bool:
620         # A split is valid only if every child node contains at least
621         # min_samples_leaf examples.
622         partitions = self._partition(X, y, attr_index)
623
624         for subset_X, _ in partitions.values():
625             if subset_X.shape[0] < self.min_samples_leaf:
626                 return False
627
628         return True
629
630     def _build_tree(
631         self,
632         X: np.ndarray,
633         y: np.ndarray,
634         attributes: list[int],
635     ) -> DecisionTreeNode:
636         # Recursively construct the modified decision tree.
637         if y.size == 0:
638             raise ValueError("Cannot build a tree from an empty dataset.")
639
640         # Stopping condition 1: all labels at the node are the same.
641         if np.all(y == y[0]):
642             return DecisionTreeNode(label=str(y[0]))
643
644         # Stopping condition 2: no attributes left to split on.
645         if not attributes:
646             return DecisionTreeNode(label=self._majority_label(y))
647
648         # Keep only attributes whose split does not create tiny child nodes.
649         valid_attributes = [
650             attr for attr in attributes if self._is_valid_split(X, y, attr)
651         ]
652
653         # Stopping condition 3: no valid split remains.
654         if not valid_attributes:
655             return DecisionTreeNode(label=self._majority_label(y))
656
657         # Among valid splits, choose the one with highest Information Gain.
658         best_attr = max(valid_attributes, key=lambda a: self._information_gain(X, y, a))
659
660         node = DecisionTreeNode(attribute_index=best_attr)
661         node.majority_label = self._majority_label(y)
662
663         partitions = self._partition(X, y, best_attr)
664         remaining_attributes = [a for a in attributes if a != best_attr]
665
666         # Recursively build child subtrees.
667         for attr_value, (subset_X, subset_y) in partitions.items():
668             child = self._build_tree(subset_X, subset_y, remaining_attributes)
669             node.children[attr_value] = child
670
671         return node

```
